In addition to the Access-Control-Allow-Methods header, the preflight response should have the following characteristics:

- *The HTTP response status should be in the 200 range.* This is defined by the CORS spec (although some browsers still process the response correctly if the status isn't in the 200 range). The sample code uses response code 204, which indicates the response is valid, but contains no body.
- *The response shouldn't have a body.* There isn't anything in the CORS spec regarding the body of the preflight response, but having a body could cause confusion for the developer, because he or she then must know how to parse and interpret the body. It's better to keep things simple and stick to the CORS headers only.
- *If a method is a simple method, it doesn't need to be listed in the Access-Control-Allow-Methods header.* (Recall from section 4.2 that the CORS spec defines simple methods as `GET`, `POST`, and `HEAD`.) If the client sends an `Access-Control-Request-Method: GET` request header, the server doesn't need to include the Access-Control-Allow-Methods header in the response. But this can be confusing. For consistency, the rest of the samples in this chapter will always include the Access-Control-Allow-Methods header, even for simple HTTP methods.

With the changes done so far, the server can now support CORS for various HTTP methods. But we aren't done with the preflight quite yet. A preflight request is also sent when the client adds additional headers to a request. The next section covers how to respond to those requests.

4.4.2 Supporting request headers with Access-Control-Allow-Headers

The previous section taught you how the server can respond to a preflight request to grant permissions to use HTTP methods. But this isn't the only type of preflight a server can receive. A browser may also send a preflight if the request contains additional HTTP headers from the client. The modified browser/server conversation is shown in figure 4.13.

TRIGGERING A PREFLIGHT FOR REQUEST HEADERS

To see this in action, let's modify the `getBlogPost` method to include a couple of additional headers when loading the blog posts. It really doesn't matter what the headers are, so let's make some up:

- *Timezone-Offset*—The user's time zone offset in minutes. This can be calculated from JavaScript.
- *Sample-Source*—The name of this book, *CORS in Action*.

The following listing adds these headers to the sample code. Again, the actual values of these headers aren't important; they are needed only to see how CORS behaves.

Listing 4.6 Adding headers to the request

```
var getBlogPosts = function() {
  var xhr = createXhr('GET', 'http://127.0.0.1:9999/api/posts');
```